CERTIFICATE PROGRAM CURRICULUM — CPC V1.0

Next.js Development:
Day 01 — Next.js Fundamentals

An intensive 12-day program — master the full Next.js 14 App Router ecosystem including server components, data fetching, authentication, SEO, middleware, caching & deployment. Today: scaffold your first app and understand how routing, layouts and components work.

"Learn. Build. Grow."

✓ React Knowledge Required ✓ 12 Live Sessions ✓ 12 Practicals ✓ Certificate on Completion ✓ Project-Based Learning

▲	○	◆	■	▣	▶
Next.js 14 App Router	**Server Components** RSC & Streaming	**Database** MongoDB & MySQL	**Auth.js** NextAuth v5	**SEO & Meta** Metadata API	**Performance** Cache & ISR

TODAY'S TOPICS AT A GLANCE

01 **Why Next.js?** React limitations & what Next.js solves at the framework level

02 **Project Setup** create-next-app, CLI options, TypeScript & Tailwind

03 **Project Structure** app/, components/, lib/, public/ & config files

04 **The App Router** File-system routing, route types & URL segments

05 **Layouts & Templates** Persistent layouts, nesting, route groups

06 **Navigation** Link, useRouter, redirect() & notFound()

07 **Server vs Client** The RSC mental model & composition pattern

▶ **Practical** Build your first multi-route Next.js application

TOPIC 01

Why Next.js? React Limitations & Next.js Advantages

React is a UI library — it gives you tools to build interactive interfaces but doesn't handle routing, data fetching, SEO, or server-side logic. As your app grows you end up stitching together dozens of packages: React Router, React Query, Helmet, Vite config, and more. Next.js solves this by providing a complete, opinionated full-stack framework built on React — with production-ready defaults that teams at Netflix, Vercel, GitHub, and Notion trust at scale.

React — What It Gives You

A library for building UI with components and JSX. You manage state, compose components, and handle interactions. Routing, SEO, and server logic are entirely your problem to solve.

Next.js — What It Adds

File-based routing, server components, built-in data fetching, API routes, image & font optimisation, middleware, and zero-config Vercel deployment.

React Limitations That Next.js Solves

REACT LIMITATION	NEXT.JS SOLUTION	WHY IT MATTERS
No routing built in	File-based App Router — folder = route	Zero config, no React Router setup, no <Routes> tree
Client-side only (CSR)	SSR, SSG, ISR & React Server Components	Pages load with HTML already — critical for SEO & performance
No SEO out of the box	Metadata API — static & dynamic meta per page	Each page exports its own title, description & OG tags
No backend / API layer	Route Handlers + Server Actions	Full-stack in one project — no separate Express server needed
JS bundle includes everything	Server Components run on server — zero JS to browser	Smaller bundle = faster load = better Core Web Vitals scores
Manual image handling	next/image — lazy load, resize, WebP/AVIF auto	Automatic optimisation, responsive sizes, zero layout shift

USE NEXT.JS WHEN

- You need SEO — blogs, marketing sites, e-commerce, portfolios
- Your app fetches data — dashboards, SaaS products, content apps
- You need a backend — auth, database, file uploads, REST APIs
- Performance matters — automatic image, font, JS optimisation

STICK WITH PLAIN REACT WHEN

- Highly interactive SPA behind login — no SEO needed at all
- You already have a separate backend & only need a pure frontend
- Mobile app — use React Native instead
- Learning React fundamentals — master React before adding Next.js

TOPIC 02

Project Setup: create-next-app, App Router & Folder Conventions

Every Next.js project starts with **create-next-app** — the official scaffolding CLI. It sets up TypeScript, ESLint, Tailwind CSS, the App Router, and the project structure in one command. Understanding folder conventions is fundamental: Next.js uses the file system as its routing API. A file at `app/about/page.tsx` automatically becomes the `/about` route — no registration needed.

Running create-next-app

```
# Scaffold a new Next.js 14 project npx create-next-app@latest my-nextjs-app # CLI prompts — recommended answers: ? Would you like to use TypeScript? Yes ? Would you like to use ESLint? Yes ? Would you like to use Tailwind CSS? Yes ? Would you like to use src/ directory? No ? Would you like to use App Router? Yes (recommended) ? Customise the default import alias (@/)? No # Start development server cd my-nextjs-app && npm run dev # Open http://localhost:3000
```

Project Structure: Folder & File Conventions

FOLDER / FILE	PURPOSE	WHEN YOU CREATE IT
app/	App Router root — all pages, layouts & API routes live here	Auto-created. Add folders for new routes
app/layout.tsx	Root layout — wraps every page. Sets <html> & <body>, global fonts, providers	Auto-created. Customise immediately
app/page.tsx	Home route (/) — the first page users see at your domain	Auto-created. Replace with your home content
app/globals.css	Global CSS — Tailwind base directives + any custom global styles	Auto-created. Add global overrides here
components/	Reusable UI components — buttons, cards, navbars, forms. Not a route.	Create manually. Group by feature or type
lib/	Utility functions, DB clients, API helpers — pure logic, no UI	Create manually. db.ts, auth.ts, utils.ts go here
public/	Static assets served from / — images, robots.txt, sitemap	Auto-created. public/logo.png → /logo.png URL
next.config.ts	Next.js config — image domains, redirects, env variables, plugins	Auto-created. Edit for custom configuration

SPECIAL FILE NAMES THE APP ROUTER RECOGNISES AUTOMATICALLY INSIDE APP/

- **page.tsx** — Makes a folder a publicly accessible route. Without it, a folder is invisible to the router
- **layout.tsx** — Wraps all pages in that folder. Persists across navigation — doesn't re-render on route change
- **loading.tsx** — Auto-shown while page loads using React Suspense. Instant loading state — zero extra code
- **error.tsx** — Catches errors in that route segment. Must be a Client Component ('use client')
- **not-found.tsx** — Renders when notFound() is thrown or a URL doesn't match any route. Branded 404.
- **route.ts** — Creates a REST API endpoint at that path. Exports GET, POST, PUT, DELETE handler functions

TOPIC 03

The App Router: Routing, Layouts & Route Groups

Next.js 14 uses a file-system router where folders define URL segments and files define UI. Every folder inside `app/` maps to a URL path segment. Creating `app/dashboard/settings/page.tsx` makes `/dashboard/settings` available instantly — no config, no imports, no registration. The App Router also supports nested layouts so each route segment can have its own persistent layout.

```
# This folder structure creates routes automatically: app/ page.tsx → / (Home) about/ page.tsx → /about dashboard/ page.tsx → /dashboard settings/ page.tsx → /dashboard/settings blog/ [slug]/ page.tsx → /blog/any-post-title (Dynamic) shop/ [...category]/ page.tsx → /shop/men/shoes/nike (Catch-all)
```

Route Types

TYPE	SYNTAX	MATCHES	USE CASE
Static	app/about/page.tsx	/about only	Fixed pages — About, Pricing, Contact
Dynamic	app/blog/[slug]/page.tsx	/blog/hello, /blog/xyz	Blog posts, product pages, user profiles
Catch-All	app/docs/[...path]/page.tsx	/docs/a, /docs/a/b/c	Docs sites, nested category trees
Optional Catch-All	app/docs/[[...path]]/page.tsx	/docs AND /docs/a/b	Docs root + all sub-pages in one file
Route Group	app/(marketing)/page.tsx	/ (folder name excluded)	Code organisation without URL impact

Layouts vs Templates vs Route Groups

LAYOUT.TSX — PERSISTENT SHELL

- **Persists across navigation** — doesn't re-render or unmount on child route changes
- **Must accept children prop** — renders the current page inside itself
- **Nestable** — dashboard/layout.tsx wraps all /dashboard/* pages
- **Server Component by default** — can fetch data directly

TEMPLATE.TSX & ROUTE GROUPS

- **template.tsx** — Re-mounts on every navigation. Use for page enter animations, analytics events
- **(folder) Route Groups** — Use (marketing), (auth) to co-locate routes without affecting URLs
- Each Route Group can have its own separate layout.tsx and loading.tsx

```
// app/dashboard/layout.tsx — Shared sidebar that persists across all /dashboard/* routes export default function DashboardLayout({ children }: { children: React.ReactNode }) { return ( <div className="flex"> <Sidebar /> // Renders once — stays mounted across /dashboard/* <main className="flex-1 p-6"> {children} // Swaps when user navigates between routes </main> </div> ) }
```

TOPIC 04

Navigation: Link, useRouter, redirect() & notFound()

Next.js provides four navigation mechanisms, each suited to different scenarios. **<Link>** is the standard declarative approach — works in any component. **useRouter** enables programmatic navigation triggered by code logic. **redirect()** handles server-side redirects before a page renders. **notFound()** throws a 404 response from inside a Server Component.

METHOD	WHERE TO USE	EXAMPLE USE CASE
<Link href="/path">	JSX in any component — Server or Client	Navbar links, buttons, card links — all declarative navigation
useRouter().push()	Client Components only ('use client')	After form submit, login success, conditional button redirect
redirect('/path')	Server Components, Server Actions, Route Handlers	Redirect unauthenticated users before page renders
notFound()	Server Components when resource is missing	DB returns null for a blog post → throw 404, show not-found.tsx

```
// <Link> — declarative (works in Server AND Client Components) import Link from 'next/link' export default function Navbar() { return ( <nav> <Link href="/">Home</Link> <Link href="/about" prefetch={true}>About</Link> </nav> ) } // useRouter — programmatic navigation (Client Components only) 'use client' import { useRouter } from 'next/navigation' export default function LoginForm() { const router = useRouter() const handleSubmit = async () => { await login() router.push('/dashboard') // Navigate after success router.refresh() // Re-fetch server data } } // redirect() — server-side (Server Components, Server Actions) import { redirect, notFound } from 'next/navigation' async function BlogPost({ params }) { const post = await getPost(params.slug) if (!post) notFound() // Shows not-found.tsx if (post.draft) redirect('/blog') // Server-side redirect return <article>{post.content}</article> }
```

TOPIC 05

Server vs Client Components — The Core Mental Model

This is the most important concept to internalise on Day 1. In the App Router, **every component is a Server Component by default**. Server Components run on the server — they send only HTML to the browser, zero JS bundle cost. Client Components are opted into with `'use client'` at the top of the file. They run in the browser and handle all interactivity, state, and browser APIs.

Server Components — Default

Run on the server at request or build time. Send only HTML — zero JS bundle cost. Can query databases directly, read env variables, access the filesystem. Cannot use useState, useEffect, event handlers, or browser APIs.

- Fetch data directly with async/await — no useEffect needed
- DB credentials stay on server — never exposed to browser
- Zero JS bundle contribution — nothing shipped to client
- Import large libraries free of bundle cost (markdown, date libs)
- Async by default — async function Page() works natively

Client Components — 'use client'

Run in the browser after hydration. Required for all user interactions, animations, and browser APIs. Add 'use client' only when you genuinely need interactivity.

- Event handlers — onClick, onChange, onSubmit, onKeyDown
- React hooks — useState, useEffect, useRef, custom hooks
- Browser APIs — window, document, localStorage, navigator
- Real-time features — WebSockets, subscriptions, timers
- Third-party UI libraries — most need 'use client' internally

The Golden Rule — Push Client Components to the Leaves

SERVER / CLIENT COMPOSITION PATTERN

- **Start with Server Components.** Only add 'use client' when you actually need interactivity, state, or a browser API.
- **Push 'use client' as far down the tree as possible.** A page can be a Server Component with one small interactive button inside.
- **Server Components can import Client Components** — but Client Components cannot import Server Components. Pass SC as children props.
- **Data fetching belongs in Server Components.** Never use useEffect + fetch in a Server Component — just async/await directly.

```
// Server Component — no 'use client', async works natively async function ProductPage({ params }: { params: { id: string } }) { const product = await getProductFromDB(params.id) // Direct DB access return ( <div> <h1>{product.name}</h1> <AddToCartButton id={product.id} /> // Client Component leaf node </div> ) } 'use client' // Only this file runs in the browser import { useState } from 'react' function AddToCartButton({ id }) { const [added, setAdded] = useState(false) return <button onClick={() => setAdded(true)}>{added ? 'Added!' : 'Add to Cart'}</button> }
```

▶ PRACTICAL

Create Your First Next.js Application

▶ BUILD A MULTI-ROUTE NEXT.JS APPLICATION FROM SCRATCH

1. **Scaffold the project.** Run `npx create-next-app@latest my-first-nextjs`. Choose: TypeScript Yes, ESLint Yes, Tailwind Yes, App Router Yes. Open the folder in VS Code and run `npm run dev`.
2. **Explore the structure.** Open `app/layout.tsx` and `app/page.tsx`. Identify: RootLayout, the children prop, the metadata export, and the default home page content. Notice there's no JSX Router anywhere.
3. **Build 3 static routes.** Create `app/about/page.tsx`, `app/services/page.tsx`, and `app/contact/page.tsx`. Add a heading and paragraph to each. Visit `/about`, `/services`, `/contact` in the browser — no config needed.
4. **Add a shared Navbar.** Create `components/Navbar.tsx`. Use `<Link href="/about">` for all navigation links. Import the Navbar into `app/layout.tsx` — it now appears on every page automatically.
5. **Create a dynamic route.** Create `app/blog/[slug]/page.tsx`. Accept `params.slug` and display it in the heading. Visit `/blog/hello-world` and `/blog/my-first-post` — both work with one file.
6. **Add a Dashboard layout.** Create `app/dashboard/layout.tsx` with a sidebar and `app/dashboard/page.tsx`. Navigate between dashboard and home — watch the sidebar persist without re-rendering.
7. **Export Metadata.** In each page.tsx add: `export const metadata = { title: 'About Us', description: '...' }`. Open DevTools → inspect <head> — see SEO tags auto-populated per page by Next.js.
8. **Client Component.** Create a counter button with useState marked `'use client'`. Import it into a Server Component page. Verify: the page stays a Server Component while the button is interactive — the correct composition pattern.

INTERVIEW QUESTIONS & MODEL ANSWERS

Day 01 — Next.js Fundamentals

Q1 What is Next.js and how does it differ from React?

React is a UI library — it handles component rendering and interactivity but provides nothing for routing, data fetching, SEO, or server logic. Next.js is a full-stack framework built on React that adds: file-based App Router, Server Components (render on server, zero JS to browser), built-in SSR/SSG/ISR data fetching, API Routes and Server Actions, Metadata API for SEO, image and font optimisation, and one-command deployment to Vercel. Next.js lets you build a complete production application without assembling a custom stack.

Q2 How does file-based routing work in the Next.js App Router?

Every folder inside app/ maps to a URL segment. A file named page.tsx makes that folder a publicly accessible route. So app/dashboard/settings/page.tsx automatically creates /dashboard/settings — no imports, no registration, no config needed. Dynamic segments use square brackets: app/blog/[slug]/page.tsx matches /blog/any-value and receives slug as a prop. Catch-all routes use [...slug]. Route Groups use (folder) — the folder name is excluded from the URL but can have separate layouts inside.

Q3 What is the difference between Server Components and Client Components?

Server Components (default in App Router) run on the server — they render to HTML and send zero JavaScript to the browser. They can directly query databases, read environment variables, and import large libraries without bundle cost. They cannot use useState, useEffect, event handlers, or browser APIs. Client Components are marked with 'use client' — they run in the browser and support all React hooks and browser APIs. Start with Server Components. Add 'use client' only when you need interactivity. Keep Client Components as leaf nodes — push them as far down the tree as possible.

Q4 What is a layout in Next.js and why does it persist across navigations?

A layout.tsx file wraps all pages in its folder and persists across navigations — it doesn’t re-render or unmount when users navigate between sibling routes. This is critical for performance: a sidebar, navbar, or auth provider in a layout renders once and stays mounted. Layouts can be nested — a root layout wraps the entire app, a dashboard layout wraps only /dashboard/* routes. Every layout must accept a children prop that renders the current page. The root app/layout.tsx must include html and body tags.

Q5 When should you use <Link> vs useRouter in Next.js?

Use <Link href="/path"> for declarative navigation in JSX — it works in both Server and Client Components and is the default for navbars, buttons, and card links. Next.js prefetches the linked route in the background for instant transitions. Use useRouter().push('/path') for programmatic navigation that must be triggered by code logic — after a form submission, authentication success, or conditional redirect. useRouter is only available in Client Components because it’s a React hook. Use redirect() from next/navigation for server-side redirects inside Server Components and Server Actions — it runs before the page renders.

NEXT

Day 02 — Routing & Layouts

Nested Routes · Dynamic Routes ([slug]) · Catch-All Routes · Route Groups (folder) · Templates vs Layouts · Practical: Multi-page Website